

Implement centralized logging with ELK Stack (Elasticsearch, Logstash, Kibana) to aggregate logs from applications and servers using terraform.

Pre Requisite:

Terraform AWS CLI

KeyPair

Launching 2 EC2 Instances

- **Node 1 (ELK Stack)** → Elasticsearch, Logstash, Kibana
- **Node 2 (App + Filebeat)** → Sample Application(node.js) + Filebeat

Using Terraform User Data to:

- Install **ELK Stack** on Node 1
- Install **Filebeat + Sample App (node.js)** on Node 2
- Configure **Filebeat to send logs to Node 1**

main.tf (Terraform Config)

This Terraform script:

- **Creates 2 EC2 instances**
- **Sets up security groups** for communication
- **Uses User Data to install ELK & Filebeat**

Create vi **main.tf** terraform file and copy the below config.

(Change key_name with your key_pair file name ie .pem or .ppk)

```
provider "aws" {  
  region = "us-east-2"  
}  
  
resource "aws_security_group" "elk_sg" {  
  name     = "elk_sg"  
  description = "Allow traffic for ELK"
```

```

ingress {
  from_port = 22
  to_port   = 22
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"] # Allow SSH
}

ingress {
  from_port = 5601
  to_port   = 5601
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"] # Allow Kibana UI
}

ingress {
  from_port = 9200
  to_port   = 9200
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"] # Allow Elasticsearch API
}

ingress {
  from_port = 5044
  to_port   = 5044
  protocol  = "tcp"
  cidr_blocks = ["0.0.0.0/0"] # Allow Filebeat Logs
}

egress {
  from_port = 0
  to_port   = 0
  protocol  = "-1"
  cidr_blocks = ["0.0.0.0/0"]
}
}

resource "aws_instance" "elk_node" {
  ami          = "ami-0cb91c7de36eed2cb" # Ubuntu AMI
  instance_type = "t2.medium"
  key_name     = "elk" # your key pair
  security_groups = [aws_security_group.elk_sg.name]
  tags = {
    Name = "elk-node"
  }
}

user_data = <<-EOF
#!/bin/bash
sudo apt update -y
sudo apt install -y openjdk-11-jre curl

# Install Elasticsearch
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add -
echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee -a
/etc/apt/sources.list.d/elastic-7.x.list
sudo apt update -y && sudo apt install -y elasticsearch

# Configure Elasticsearch
echo "network.host: 0.0.0.0" | sudo tee -a /etc/elasticsearch/elasticsearch.yml
echo "discovery.type: single-node" | sudo tee -a /etc/elasticsearch/elasticsearch.yml
sudo systemctl enable elasticsearch && sudo systemctl start elasticsearch

# Install Kibana
sudo apt install -y kibana
echo "server.port: 5601" | sudo tee -a /etc/kibana/kibana.yml

```

```

    echo "server.host: 0.0.0.0" | sudo tee -a /etc/kibana/kibana.yml
    echo "elasticsearch.hosts: [\"http://localhost:9200\"]" | sudo tee -a
/etc/kibana/kibana.yml
    sudo systemctl enable kibana && sudo systemctl start kibana

```

```

# Install Logstash
sudo apt install -y logstash
echo '
input {
  beats {
    port => "5044"
  }
}
filter {
  json {
    source => "message"
  }
}
output {
  elasticsearch {
    hosts => ["http://localhost:9200"]
    index => "logs-%%{+YYYY.MM.dd}"
  }
}
| sudo tee /etc/logstash/conf.d/logstash.conf
sudo systemctl enable logstash && sudo systemctl start logstash

echo "ELK stack setup completed!"
EOF
}

```

```

resource "aws_instance" "app_node" {
  ami          = "ami-0cb91c7de36eed2cb" # Ubuntu AMI
  instance_type = "t2.micro"
  key_name     = "elk" # your key pair
  security_groups = [aws_security_group.elk_sg.name]
  tags = {
    Name = "app-node"
  }
}
user_data = <<-EOF
#!/bin/bash
sudo apt update -y

```

```

# Install Filebeat
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo apt-key add -
echo "deb https://artifacts.elastic.co/packages/7.x/apt stable main" | sudo tee -a
/etc/apt/sources.list.d/elastic-7.x.list
sudo apt update -y && sudo apt install -y filebeat

```

```

# Configure Filebeat
echo '
filebeat.inputs:
- type: log
  enabled: true
  paths:
    - /var/log/sample_app.log

```

```

# Capture system logs
- type: log
  enabled: true
  paths:
    - /var/log/syslog # Ubuntu system logs
    - /var/log/messages # Amazon Linux system logs

```

```

output.logstash:
  hosts: ["${aws_instance.elk_node.private_ip}:5044"]
' | sudo tee /etc/filebeat/filebeat.yml

sudo systemctl enable filebeat && sudo systemctl start filebeat

# Install Sample Node.js App
sudo apt install -y nodejs npm
mkdir /home/ubuntu/app && cd /home/ubuntu/app
echo 'console.log("Sample App Running"); setInterval(() => console.log("Log Entry"),
5000);' > app.js
node app.js > /var/log/sample_app.log 2>&1 &
EOF
}

output "elk_ip" {
  value = aws_instance.elk_node.public_ip
}

output "app_ip" {
  value = aws_instance.app_node.public_ip
}

output "elk_ssh" {
  value = "ssh -i my-key.pem ubuntu@${aws_instance.elk_node.public_ip}"
}

output "app_ssh" {
  value = "ssh -i my-key.pem ubuntu@${aws_instance.app_node.public_ip}"
}

```

Deploy the Infrastructure on AWS.

```

terraform init
terraform plan
terraform validate
terraform apply -auto-approve

```

Check Outputs (SSH Commands & IPs) `terraform output`

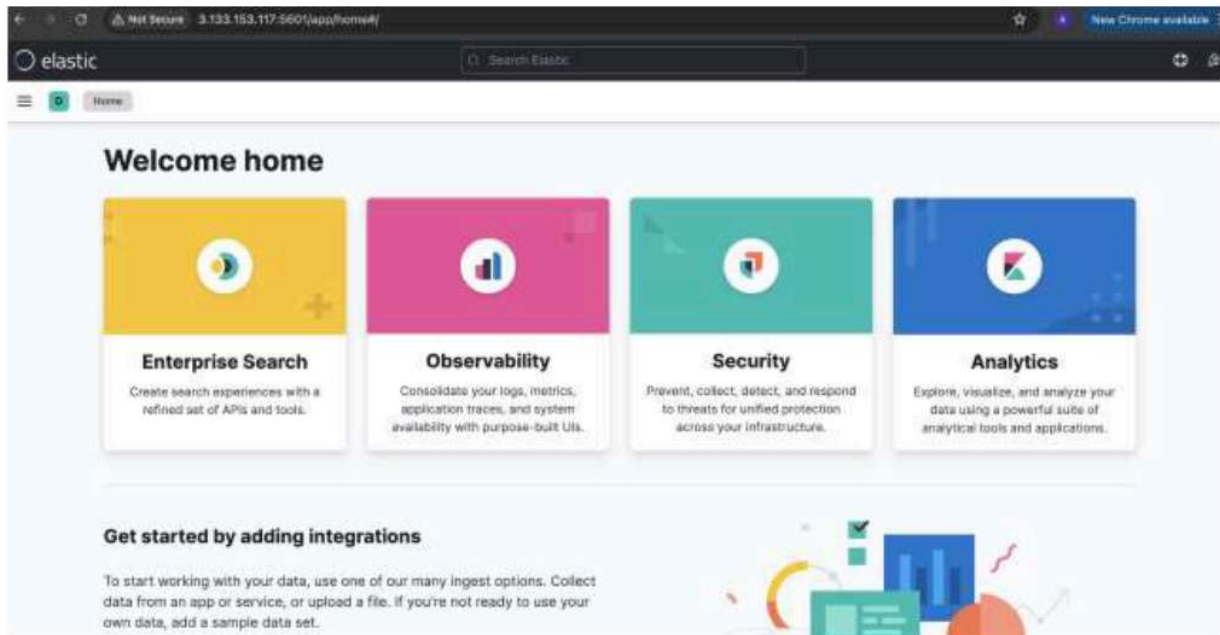
Filebeat on Node 2 will collect:

- **Application Logs (Node.js)**
- **System Logs (CPU, Memory, etc.)**
- **Send everything to Logstash on Node 1**

Verify the Setup

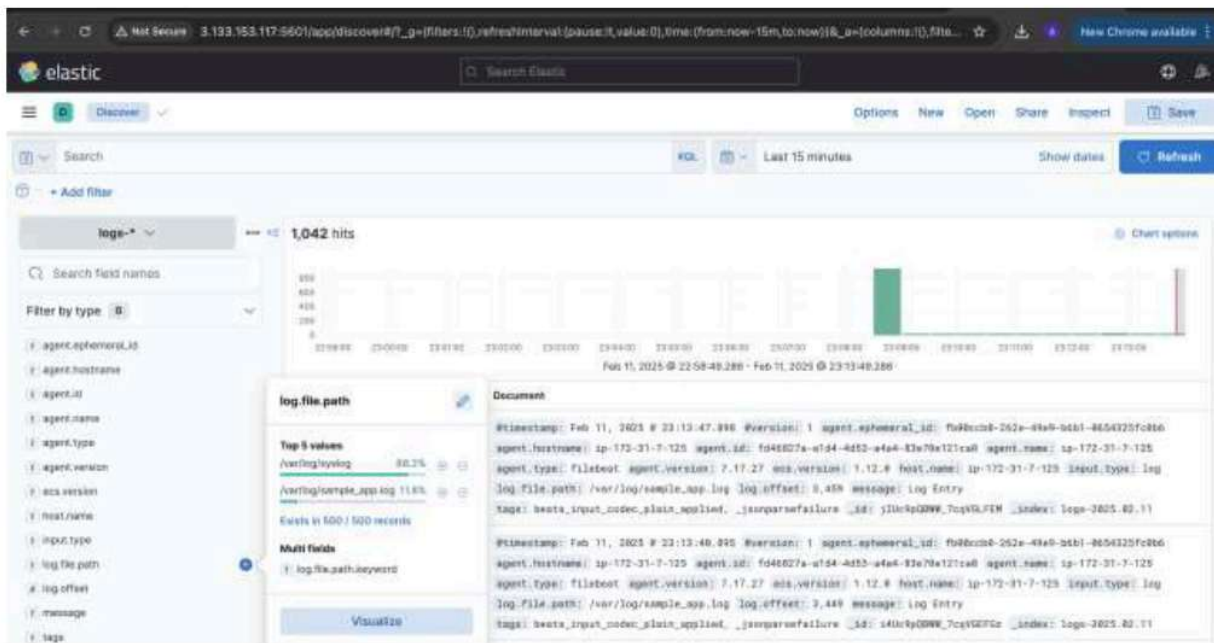
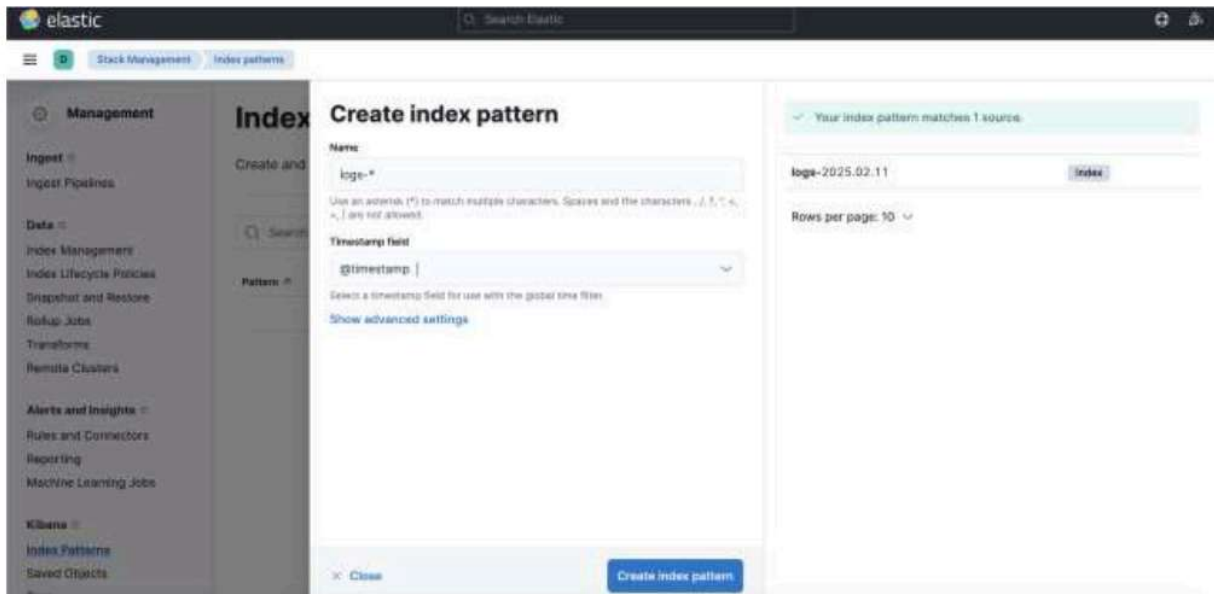
- **ELK Dashboard (Kibana)**

`http://<ELK_NODE_PUBLIC_IP>:5601` (it will take 1-2min to load mostly or show an error incase)



Check Logs in Kibana Discover

1. Go to **Kibana > Discover**
2. Select **logs-*** index
3. Search for logs



Now, let destroy the infrastructure what we have created using

terraform destroy --auto-approve

.....

If you face any error, let troubleshoot the issue:

SSH into ELK & App Node

```
ssh -i my-key.pem ubuntu@<ELK_NODE_PUBLIC_IP>  
ssh -i my-key.pem ubuntu@<APP_NODE_PUBLIC_IP>
```

Issue mostly with configuration of elastic search, logstash or kibana.

Some time due to some network glitch setup miss to update properly so,
manually logging into the ELK node and try checking the status of ELK.

Check status of Elasticsearch, Logstash, Kibana

```
java -version
```

If java is missing, install it:

```
sudo apt update && sudo apt install -y openjdk-11-jdk
```

```
sudo systemctl status elasticsearch
```

```
sudo systemctl status logstash
```

```
sudo systemctl status kibana
```

If any issue with any service perform below steps:

```
sudo systemctl enable elasticsearch
```

```
sudo systemctl restart/start elasticsearch
```

```
sudo systemctl status elasticsearch
```

```
sudo systemctl enable kibana
```

```
sudo systemctl start/restart kibana
```

```
sudo systemctl status kibana
```

```
sudo systemctl enable logstash
```

```
sudo systemctl restart/start logstash
```

`sudo systemctl status logstash`

If logstash not installed > use below command to install

`sudo apt-get install logstash -y`

If Elasticsearch still doesn't start properly, check logs:

`sudo journalctl -u elasticsearch --no-pager | tail -50`

ElasticSearch: Open the Elasticsearch config file:

`sudo vi /etc/elasticsearch/elasticsearch.yml`

Make sure the following lines is added:

```
network.host: 0.0.0.0
discovery.type: single-node
```

```
path.data: /var/lib/elasticsearch
#
# Path to log files:
#
path.logs: /var/log/elasticsearch
#
# ----- Memory -----
#
# Lock the memory on startup:
#
bootstrap.memory_lock: true
#
# Make sure that the heap size is set to about half the memory available
# on the system and that the owner of the process is allowed to use this
# limit.
#
# Elasticsearch performs poorly when the system is swapping the memory.
#
# ----- Network -----
#
# By default Elasticsearch is only accessible on localhost. Set a different
# address here to expose this node on the network:
#
network.host: 0.0.0.0
#
# By default Elasticsearch listens for HTTP traffic on the first free port it
# finds starting at 9200. Set a specific HTTP port here:
#
http.port: 9200
#
# For more information, consult the network module documentation.
#
# ----- Discovery -----
#
# Pass an initial list of hosts to perform discovery when this node is started:
# The default list of hosts is ["127.0.0.1", "[:,1]"]
#
discovery.seed_hosts: []
discovery.type: single-node
# Bootstrap the cluster using an initial set of master-eligible nodes:
#
cluster.initial_master_nodes: ["node-1", "node-2"]
#
# For more information, consult the discovery and cluster formation module documentation.
#
```


After making these changes, check if Elasticsearch is running:

```
sudo systemctl restart elasticsearch
```

```
sudo systemctl status elasticsearch
```

You can also the health of the elasticsearch using curl.

```
curl -X GET
```

```
"http://localhost:9200/_cluster/health?pretty"
```

```
ubuntu@ip-172-31-2-8:~$ curl -X GET "http://localhost:9200/_cluster/health?pretty"
{
  "cluster_name" : "elasticsearch",
  "status" : "yellow",
  "timed_out" : false,
  "number_of_nodes" : 1,
  "number_of_data_nodes" : 1,
  "active_primary_shards" : 10,
  "active_shards" : 10,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 1,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 90.9090909090909
}
```

Kibana Config: `sudo vi /etc/kibana/kibana.yml`

Need to make sure config should be added:

```
server.port: 5601
```

```
server.host: "0.0.0.0"
```

```
elasticsearch.hosts: ["http://localhost:9200"]
```

```
sudo systemctl start/restart kibana
```

```
sudo systemctl status kibana
```

```
# Kibana is served by a back end server. This setting specifies the port to use.
server.port: 5601

# Specifies the address to which the Kibana server will bind. IP addresses and host names are both valid values
# The default is 'localhost', which usually means remote machines will not be able to connect.
# To allow connections from remote users, set this parameter to a non-loopback address.
server.host: "0.0.0.0"

# Enables you to specify a path to mount Kibana at if you are running behind a proxy.
# Use the 'server.rewriteBasePath' setting to tell Kibana if it should remove the basePath
# from requests it receives, and to prevent a deprecation warning at startup.
# This setting cannot end in a slash.
```

Summary :

Now you can **visualize failed transactions, errors, and patterns!**

```
#logging.verbose: false

# Set the interval in milliseconds to sample system and process performance
# metrics. Minimum is 100ms. Defaults to 5000.
#ops.interval: 5000

# Specifies locale to be used for all localizable strings, dates and number formats.
# Supported languages are the following: English - en , by default , Chinese - zh-CN .
#i18n.locale: "en"
#server.host: 0.0.0.0
elasticsearch.hosts: ["http://localhost:9200"]
```

**ELK fully configured & logs indexed in Elasticsearch View
logs in Kibana UI**

Correlate application logs with transaction traces

Fully Automated ELK Stack using Terraform